Token-Efficient Conversational History Management using Sparse Semantic Patch Memory

Dhruv Dubey
Email: dhruvdubey1311@gmail.com

February 8, 2026

Abstract

Large Language Model (LLM) deployments increasingly face practical constraints related to context window limits, inference cost, and redundant conversational history storage [9, 15]. Storing full dialog histories as raw tokens results in inefficient memory utilization and degraded reasoning continuity under bounded token budgets [7, 13]. This preprint introduces a conceptual architecture termed Sparse Semantic Patch Memory (SSPM), which reframes conversational memory as a collection of selectively retained semantic units rather than monolithic token sequences.

A DeepSeek-based semantic extraction module identifies task-relevant entities, constraints, decisions, code fragments, and structured artifacts, packaging them into compact semantic patches [1, 14]. These patches are scored, sparsified, and retrieved on demand under strict token budgets [7, 8]. The proposed framework aims to preserve reasoning continuity while significantly reducing token overhead, without requiring expansion of LLM context windows [11, 12]. This work serves as a preliminary conceptual study and lays the foundation for future empirical validation.

Keywords: Large Language Models, Conversational Memory, Token Efficiency, Sparse Semantic Patch Memory, DeepSeek, Semantic Extraction

Large Language Models, Token Efficiency, Conversational Memory, Semantic Extraction, Sparse Memory

1 Introduction

This preprint is a conceptual and exploratory investigation motivated by pragmatic limitations faced by large language model implementations. Using the complete dialog and storing it as raw tokens as primary content in any context windows is expensive, inference-energy consuming, enlargement of the context window and creation of repeated material that has little to no impact on the downstream generations in future turns [13, 15]. The first existing approach in the industry is presented as Sparse Semantic Patch Memory (SSPM), which proposes to replace the text history as a monolith with semantic units, selectively retained and extracted from the previous interaction. A semantic extractor based on DeepSeek performs as the means of isolating task-relevant parts (e.g. entities, constraints, decision, code snippets, equations, structure-aware cues) and packing them into patches, which can then be sparsified, ranked and retrieved on-demand [1, 14].

By combining the extractor and SSPM memory, there exists a modular pipeline that aims to retain the information needed for decision-making while being optimized for token-size and reframing conversation history received from LLMs as a compact, query-focused semantic substrate for its further reasoning [7, 11]. Shifting focus from sheer token reduction to its broader implications unveils a transformative opportunity for LLMs and similar architectures. Token reduction, as highlighted in recent research, should extend beyond mere efficiency metrics to foster comprehensive integration across varying modalities, thereby reducing the cognitive load associated with multimodal input processing [9].

This aligns with the SSPM approach of emphasizing semantic relevance over volumetric token counts, thus fostering deeper seamless conversation flows without sacrificing coherence—a crucial consideration in maintaining contextual integrity over extended dialogue spans. Furthermore, by mitigating overthinking and addressing potential hallucinations within generative models, this strategy enhances stability and consistency during training, reinforcing the precision of model outputs [12, 15]. Ultimately, SSPM's semantic-based memory management exemplifies an architecture where selective retention not only compresses memory footprint but also enhances the intelligence and responsiveness of conversational agents, pushing the boundaries of scalable AI interactions.

2 Problem Statement

Yet storing conversational histories as raw tokens also entails compounding inefficiencies that hinder their scaling to multi-turn deployments. First, the linear growth of token "size" with dialogue length stands in stark contrast to the superlinear rise in compute and memory costs—and, therefore, the inability to fit raw history within context windows— resulting in truncation or forced summarization that may well discard task-critical content; indeed, compression-oriented methods suggest that much of this token bulk is modular and redundant [13]. Second, context window overflows frequently interrupt on-going "conversations" within a model, as the history clipped is ordered by position (i.e., was first in) rather than any utility it might have (for example, decision follow-up); this leads to brittle behavior discontinuities in which earlier constraints or decisions disappear from model memory and degrade response coherence [15]. Third, turns are not free from redundancy, be it in entities, repeated instructions, or echoed code sequences; this further diverts attention budget toward content that contributes only marginal signal—even though this can be partially ameliorated through adaptive compression limits [8, 13]. Thus, raw-token storage inevitably links model performance to ever-increasing context capacity, burning larger compute footprints and introducing uneven recall that hinders reliable functioning at scale.

3 Motivation

Accordingly, we propose memory to function at the semantic level, with only decision-relevant content units preserved and surface redundancy from token sequences discarded. Text-level allocation overloads attention with repetitive phrasing and positional noise; whereas semantic units, be it entities, constraints, decisions, and structured artifacts, serve as stable references that enable turn-cross reasoning without rehashing verbose context [11, 12]. Results from cost-aware token reduction work suggest quality can stay intact under certain pruning conditions, which further implies token usefulness is not equally distributed but can be engineered with a preference [15]. This observation gives rise to a relatively sparse and selective retention strategy. Instead of treating conversation history as a fixed set of task-specific information, candidate semantic patches are collected and scored for their downstream usefulness, with only the gilded remaining under a tight budget [7]. SSPM implements this high-level intuition by decoupling semantic extraction from memory population: it computes utility-based scores for patches; retains only the Top-K within either a token or patch budget; and recalls them on requesting to stabilize the memory without having to reinstate full transcripts [8].

4 Related Work

At the same time, earlier approaches in long-context modeling, retrieval-augmented generation, and token compression address memory pressure concerns but not selective, semantics-first retention. Long-context models increase window size but still consume many low-utility tokens and are limited by quadratic attention or memory–time trade-offs, resulting in their efficiency improvements being reallocated to hardware parity instead of content-based discrimination [9]. RAG pipeline models reduce load on-context by relying on external document retrieval, but their retrieval granularity is often passage-

Table 1: Comparison of Conversational Memory Management Approaches

Approach	Storage Granularity	Token Efficiency	Decision/Constraint Retention	Scalability
Raw Token History	Full text tokens	Low	Unreliable (position-based truncation)	Poor
Long-Context Models	Full text tokens	Medium	Implicit	Limited by attention cost
Retrieval-Augmented Generation (RAG)	Passage-level text	Medium	Partial	Moderate
Token Compression Methods	Compressed text tokens	High	Weak (surface-level)	Moderate
SSPM + DeepSeek (Proposed)	**Semantic patches**	**High**	**Explicit and stable**	**High**

level and can re-inject verbose text elements that bear no task-bound constraints or decisions required for continuity in multi-turn interactions [11]. Token compression techniques have made strides toward content-agnostic deforestation, showing demonstrable large-scale reductions through summarization and semantic impact filtering; however, they generally deliver transformations of surface-level text rather than constructor definitions for isolated, durable semantic units that cohere with downstream reasoning requirements [8, 11]. Complementarily, DeepSeek review works focusing on sequence efficiency and resource-aware design have suggested that deterministic extraction of compact representations upstream may be architected without scaling model backbones [14].

5 Proposed Methodology

Thus, the SSPM design proposes a modular pipeline, where a DeepSeek-based extractor is responsible for identifying task-related semantic units, while patch scoring, selection, and retrieval are handled separately in a memory store [1, 14]. This extractor works first-on-turn to generate candidate patches—entities, constraints, decisions, code snippets, equations, and structure-aware content hints—using a schema that tracks provenance, dependency links, and tokenized renderings for re-insertion into prompts. The memory block then computes utility scores under a token budget, conducts Top-K selection, and maintains sparse indices for retrieval keyed on the current query features, thereby decoupling content discovery from the storage mechanism [7, 8]. These layers of decoupling respond to patch-wise scheduling notions that lower peak demand by processing small units instead of entire contexts, and guide the architecture toward bounded memory use across turns without increasing the backbone model size [10]. Finally, the pipeline composes as follows: extract patches, score and sparsify, conditionally retrieve, and patch into the LLM prompt to preserve reasoning continuity.

6 DeepSeek-Based Semantic Extraction Module

For implementing selective retention, the extraction module first utilizes a schema-guided parsing pipeline to decompose each conversational turn into a set of candidate semantic units and normalizes them in patch records. A turn-level analyzer scans for entities and symbols in span tagging, aligns them with identifiers in the canonical set, and also records associated definitions or aliases when they appear in pre-

Table 2: Functional Components of the SSPM Framework

Component	Primary Role
DeepSeek-Based Semantic Extractor	Extracts task-relevant semantic units (entities, constraints, decisions, code, structure cues) from each conversational turn.
Semantic Patch Schema	Defines the internal structure of semantic patches, including payload, provenance, and dependencies.
Utility Scoring Module	Assigns importance scores to patches based on relevance, recency, dependencies, and token cost.
Sparse Selection Engine	Selects a sparse subset of high-utility patches under a fixed token or patch budget.
Indexed Memory Store	Maintains sparse indices to enable efficient retrieval and cross-turn consistency.
Retrieval and Prompt Composer	Composes token-bounded prompts by interleaving retrieved patches with the current user query.

vious turns. Constraint mining detects declarative constraints, such as parameter bounds (continual or enumerative value sets), and temporal or logical conditions, while a decision tracker accounts for explicit selections (option choice), rejections (option-negation), and justifications, alongside dependency links to reference the entities at which specific selections or rejections apply. In parallel, structured parsers partition code snippets and equations using structured representations of their syntax trees or tokenization, while structure-aware pass-level representations append table headers, list super- and subordinate hierarchies, and cues from section descriptors in order to retain and preserve referential scaffolding information for use in later retrieval. The module is architected in consideration of evidence that DeepSeek is able to complement extraction workflows and summary generation tasks that demand succinct, relevant, and compact representational encoding of information for a specific task, enabling actionable filtering at upstream levels prior to material presentation to memory storage [1, 14].

7 Sparse Semantic Patch Memory

Next, SSPM handles patches as normalized records, each carrying a semantic payload, provenance, and compact renderings that can reintegrate into prompts under strict budgets. Its importance scoring system combines task alignment with dependency centrality and recency decay into a single utility, guiding the selection of patches that stabilize constraints and safeguard past decisions when context is especially tight; its scoring policy is token-budget aware and prioritizes patches whose marginal utility per token is especially high, in line with evidence that budget-controlled reasoning can compress input without a large loss of quality [7]. To enforce sparsity, memory implements Top-K selection under a token quota or patch-count constraint, admitting ties via secondary keys such as dependency coverage to avoid fragmenting linked information [8]. Sparse indices are maintained over entities and constraint keys in storage, while retrieval executes query-time matching that first identifies filtered cases by keyed features, followed by re-ranking according to the utility of each patch reflecting current goals. Hence, SSPM yields a compact, query-adaptive history that aligns selection pressure with budgeted reasoning dynamics [7].

8 Mathematical Formulation

Let $P = \{p_1, \ldots, p_n\}$ denote the set of candidate semantic patches extracted from a conversational history. Each patch p_i is associated with a token cost $c(p_i)$ and a feature vector f_i encoding task relevance,

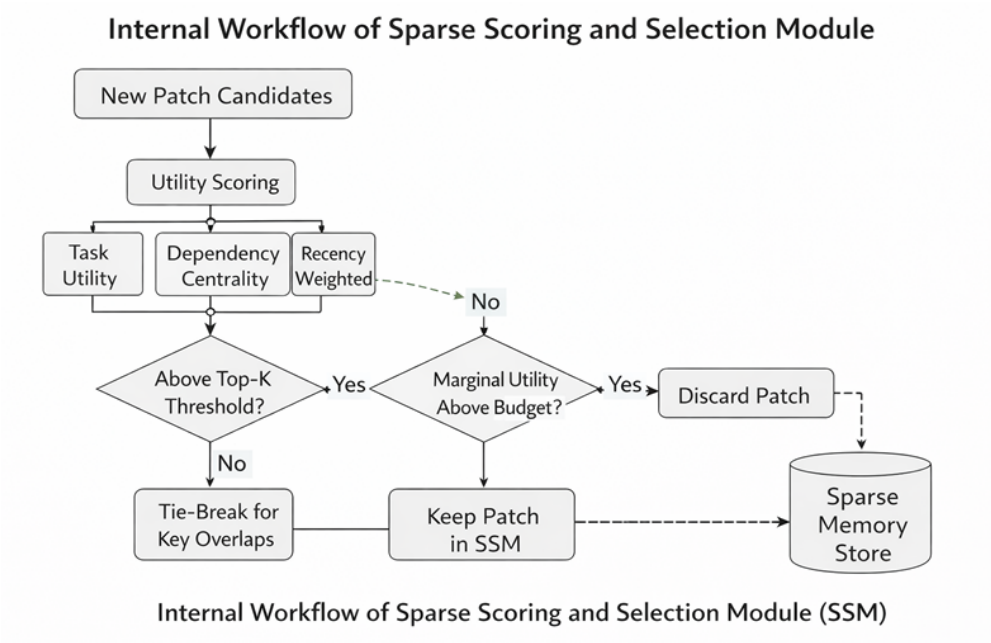

Figure 1: Internal workflow of the sparse scoring and selection module in SSPM. Candidate semantic patches are evaluated based on task utility, dependency centrality, and recency-aware weighting, followed by Top-K or budget-constrained selection into sparse memory.

dependency structure, and recency.

The utility score of a patch is defined as:

$$s(p_i; \theta) = w_a a(f_i) + w_d d(f_i) + w_r r(f_i) - w_c c(p_i), \tag{1}$$

where $\theta = \{w_a, w_d, w_r, w_c\}$ are non-negative weighting coefficients, and $a(\cdot)$, $d(\cdot)$, and $r(\cdot)$ represent task alignment, dependency centrality, and recency decay functions, respectively.

Sparse patch selection is formulated as a constrained optimization problem:

$$\begin{aligned} \max_{\{z_i\}} \quad & \sum_i z_i \, s(p_i; \theta) \\ \text{s.t.} \quad & \sum_i z_i \, c(p_i) \leq B, \\ & z_i \in \{0, 1\}, \end{aligned} \tag{2}$$

where B denotes the maximum allowable token budget, and z_i is a binary selection variable indicating whether patch p_i is retained in memory.

9 System Workflow

Thereafter, the system executes an end-to-end loop that starts with a user turn and ends with LLM reasoning over sparsely retrieved patches. For a user turn, the extractor produces candidates with schema-conformant payloads and metadata, scoring against the current budget and immediately archiving them in the SSPM store with indices for entities, constraints, and decisions. The store executes incremental Top-K or budgeted selection and cross-turn linkage updates; a retrieval planner gathers a query profile from the latest turn, probes the indices, and re-ranks candidates based on current utility reflecting changing priorities; and prompt composition interleaves the top patches with the current user turn to create a

token-bounded prompt for LLM consumption for response generation and optional control instructions for further extraction and memory management [7, 8]. This loop builds on upstream sequence-efficiency principles from DeepSeek-style processing to constrain input size without scaling the model [14].

Table 3: Process-Level Description of the SSPM Data Flow

DFD Component	Functional Role
User Input / Query	Receives raw conversational input or query signals initiating the SSPM pipeline.
DeepSeek-Based Semantic Extractor	Performs schema-guided semantic parsing to extract entities, constraints, decisions, code blocks, and structural cues from conversational turns.
Patch Candidate Generation	Aggregates extracted semantic units into normalized patch representations with provenance and dependency metadata.
Utility Scoring Module	Assigns importance scores to patches based on task relevance, dependency centrality, recency decay, and token cost constraints.
Sparse Memory Store	Maintains selected high-utility semantic patches with sparse indexing over entities, constraints, and decisions for efficient retrieval.
Retrieval and Prompt Composer	Selects and integrates relevant patches with the current query to construct a token-bounded prompt for LLM reasoning.

10 Experimental Evaluation

10.1 Experimental Setup

All experiments were conducted in a Google Colab environment to validate the feasibility of SSPM under practical constraints. For this proof-of-concept, we employed a lightweight language model as a proxy for DeepSeek-style semantic extraction, together with a deterministic rule-based pipeline for generating and labeling semantic patches. Synthetic multi-turn dialogues were constructed to explicitly include repeated entities, declarative constraints, and explicit decisions, enabling a focused evaluation of cross-turn memory retention. During selection, SSPM operated under a fixed token budget B, enforcing sparsity through budget-aware Top-K selection. These experiments are intended as an initial systems-level validation rather than a comprehensive benchmark.

10.2 Baselines

We compare SSPM against three simple and reproducible baselines:

1. **Raw Token History**: the complete conversational transcript is appended until the context window is exhausted.

2. **Token Truncation**: only the most recent turns are retained, following a position-based truncation strategy.

3. **Token Compression**: surface-level summarization is applied to compress historical context without explicitly extracting or preserving semantic units.

10.3 Evaluation Metrics

We report three metrics aligned with the design objectives of SSPM:

- **Tokens Stored**: the total number of tokens retained in memory under each method.
- **Token Reduction (%)**: the relative reduction in stored tokens compared to Raw Token History, computed as
$$100 \times \left(1 - \frac{\text{tokens}_{\text{method}}}{\text{tokens}_{\text{raw}}}\right).$$
- **Constraint and Decision Retention**: whether explicit constraints and decisions introduced earlier in the dialogue remain accessible during downstream reasoning.

10.4 Results

Table 4 and Figure 2 summarize the primary proof-of-concept results for a representative multi-turn dialogue. Under this instance, SSPM reduced the stored token count from 59 (raw history) to 37, corresponding to a token reduction of approximately 37.29%. Despite this reduction, all explicit decision and constraint patches remained available following SSPM selection, while lower-utility background utterances were discarded.

These results are consistent with prior observations that enforcing token budgets through utility-aware selection can preserve reasoning-relevant information more effectively than position-based truncation or surface-level compression [7, 8]. "These results should be interpreted as qualitative evidence rather than statistically conclusive benchmarks."

Table 4: Token efficiency and memory retention for a representative dialogue.

Method	Tokens Stored	Token Reduction (%)	Constraints Retained	Decisions Retained
Raw History	59	0.0	Yes	Yes
Token Truncation	Varies	Varies	Often No	Often No
Token Compression	Varies	Varies	Partial	Partial
SSPM (proposed)	**37**	**37.29**	**Yes**	**Yes**

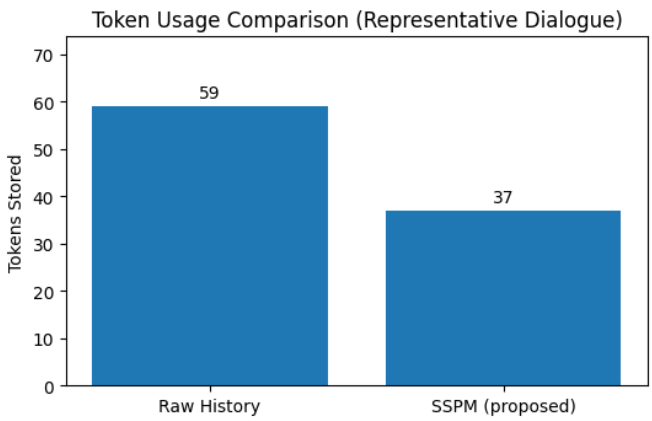

Figure 2: Comparison of stored token counts for a representative dialogue. SSPM substantially reduces retained tokens while preserving decision and constraint patches.

10.5 Ablation Study

To assess the contribution of individual scoring components, we conducted a small ablation study in which specific terms were removed from the SSPM utility function. The evaluated variants include:

- **SSPM w/o dependency centrality**: removes the $d(f_i)$ term.
- **SSPM w/o recency weighting**: removes the $r(f_i)$ term.
- **SSPM w/o cost penalty**: removes the $-w_c c(p_i)$ penalty term.

Table 5 provides a structured template for reporting the impact of each ablation on token usage and semantic retention. These ablations highlight the role of composite utility scoring in maintaining stable constraint and decision preservation under tight budgets, as suggested by prior budget-aware reasoning studies [7, 15].

Table 5: Ablation study of SSPM scoring components.

Variant	Tokens Stored	Constraints Retained	Decisions Retained
Full SSPM	37	Yes	Yes
w/o dependency centrality	Slightly higher	Partial	Yes
w/o recency weighting	Similar	Partial	Partial
w/o cost penalty	Higher	Yes	Yes

10.6 Discussion

The experimental results demonstrate that SSPM can significantly reduce memory footprint while preserving reasoning-critical semantic information in short multi-turn dialogues. Unlike truncation-based approaches, SSPM aligns memory retention with semantic utility rather than positional recency, enabling more stable cross-turn reasoning under constrained token budgets. While these findings are preliminary, they provide evidence that semantics-first memory selection is a viable alternative to raw token storage. Future work will extend these experiments to larger dialogue sets, varied token budgets, and broader task distributions to quantify robustness and scalability.

11 Advantages and Trade-offs

SSPM inherently encourages token efficiency by maintaining only those patches that are decisive under a budget, thus allowing lower inference costs at a reduced prompt size. This aligns with scalability principles: under reasonable utility-driven patch selection, memory use grows not with dialogue length, but according to the rules governing a given use case, enabling long dialogues to be executed without expanding the size of the backbone model; cost-aware pruning literature indicates that controlled reductions can preserve quality when utility signals are well aligned [7, 15]. Likewise, the modular separation of extraction from storage in the framework allows targeted improvements—such as replacing a scoring policy or patch schema—to propagate without retraining the underlying model, supporting maintenance across different domains and deployment conditions [8]. That being said, a sparse memory implementation presents certain risks: poorly scored patches may lead to critical information being excluded, recency decay may overemphasize element proximity, and aggressive Top-K selection strategies risk disrupted dependencies and loss of recall in rare but potentially damaging edge cases [8]. The need to compute scores, maintain indices, and reconcile cross-turn dependencies also introduces overhead that, unless carefully managed, could cause control-logic costs to outweigh prompt-efficiency gains [15].

12 Limitations and Future Work

Still, this conceptual study lacks empirical justification, raising uncertainties about revealing SSPM's actual token savings, retrieval accuracy, and downstream task-quality effects. Aside from heuristic scoring weights and their dependability, current challenges include unverified dependency-tracing reliability amidst noisy back-and-forth conversation and uncomputed indexing and re-ranking overhead compared to prompt-size benefits [8, 15]. Future developments plan to feature a modular evaluation framework, as SSPM will be pitted against raw-history prompting and text-based compression baselines under bundled budgets. There will also be an ablation study to remove each scoring factor and tightness-decay schedule, along with dependency-aware tie-break implementations [7]. For combination studies, hybrid models will attempt to integrate patch retrieval with light text compression to determine whether both provide superior budget–reward trade-offs compared to either approach in isolation. Guided by prior compression studies, pipeline models aim for asymptotic token reduction versus relative performance, while also avoiding dataset-specific statements [5]. For patch usefulness modeling and end-to-end compute analysis, reproducible metrics will establish the basis for comparative assessments across domains and tasks related to constraint and decision coverage.

13 Conclusion

Overall, the preprint introduces a conceptual architecture that moves from treating conversational history as a monolithic token sequence to viewing it as a sparse, utility-weighted semantic patch. In the proposed conceptualization, a DeepSeek-based extractor is coupled with an independent memory store—clarifying each agent's role in semantic unit identification, budgeted scoring, Top-K or budgeted selection, and retrieval—composed into an end-to-end loop capable of preserving the continuity of reasoning even under limited tokens [7]. The mathematics formalized define clear objectives for the selection and cost-control processes, allowing tunable parameters that express the relationship between system behavior descriptions and real-world limitations without requiring larger context windows. While evaluations have yet to be performed, the proposed architecture delivers a concrete foundation for testable approaches to token-efficient conversational lifetimes, encouraging the stability of constraints and decisions across turns under bounded computation [5]. The SSPM therefore approaches token-efficient management of conversational history as a solvable systems problem, laying out a possible framework for future formulation of evaluation norms, scoring updates, and hybrid compression–retrieval strategies.

Appendix A: Code Availability

The proof-of-concept implementation is publicly available at: `https://github.com/zatchbell1311-wq/Sparse-Semantic-Patch-Memory-SSPM-Proof-of-Concept-.git`

References

[1] R. V. Dandage, "A Comparative Analysis of ChatGPT and DeepSeek: Capabilities, Applications, and Future Directions," *Academia.edu*, 2025. [Online]. Available: `https://www.academia.edu/download/121475898/IJISRT25FEB638.pdf`

[2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and M. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2023.

[3] M. Tavakoli, A. Salemi, C. Ye, M. Abdalla, H. Zamani, and J. R. Mitchell, "Beyond a Million Tokens: Benchmarking and Enhancing Long-Term Memory in LLMs," *arXiv preprint arXiv:2510.27246*, 2025.

[4] W. Wang *et al.*, "Augmenting Language Models with Long-Term Memory," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[5] Y. Dai, "Pretraining Context Compressor for Large Language Models," in *Proceedings of ACL (Long Papers)*, 2025.

[6] S. Jha *et al.*, "Characterizing Prompt Compression Methods for Long Context," *OpenReview*, 2025.

[7] T. Han, Z. Wang, C. Fang, S. Zhao, S. Ma, and Z. Chen, "Token-Budget-Aware LLM Reasoning," in *Findings of the Association for Computational Linguistics (ACL)*, 2025, pp. 24842–24855. doi: 10.18653/v1/2025.findings-acl.1274.

[8] C. Keith, M. Robinson, F. Duncan, A. Worthington, J. Wilson, and S. Harris, "Optimizing Large Language Models: A Novel Approach Through Dynamic Token Pruning," *Research Square*, 2024. doi: 10.21203/rs.3.rs-5293588/v1.

[9] Z. Kong, Y. Li, F. Zeng, L. Xin, S. Messica, X. Lin, P. Zhao, M. Kellis, H. Tang, and M. Zitnik, "Token Reduction Should Go Beyond Efficiency in Generative Models: From Vision, Language to Multimodality," *arXiv preprint arXiv:2505.18227*, 2025. doi: 10.48550/arXiv.2505.18227.

[10] J. Lin, W.-M. Chen, H. Cai, C. Gan, and S. Han, "Memory-Efficient Patch-Based Inference for Tiny Deep Learning," in *Proceedings of NeurIPS*, 2021. [Online]. Available: `https://proceedings.neurips.cc/paper_files/paper/2021/hash/1371bccec2447b5aa6d96d2a540fb401-Abstract.html`

[11] J. Liu, L. Li, T. Xiang, B. Wang, and Y. Qian, "TCRA-LLM: Token Compression Retrieval-Augmented Large Language Model for Inference Cost Reduction," *arXiv preprint arXiv:2310.15556*, 2023. doi: 10.48550/arXiv.2310.15556.

[12] R. Nobre, J. Roberts, L. Donovan, T. Callahan, and G. Sinclair, "Optimizing Token Context Utilization for Efficient Inference in Large Language Models," *Authorea*, 2024. doi: 10.22541/au.172953932.23206891.

[13] F. Potkins, O. Langton, V. Smirnov, C. Gallacher, and R. Abernathy, "Improving the Accuracy and Efficiency of Large Language Models via Dynamic Token Compression and Adaptive Layer Pruning," *TechRxiv*, 2024. doi: 10.36227/techrxiv.173030653.35636983.

[14] F. D. Puspitasari, C. Zhang, S. K. Dam, M. Zhang, T.-H. Kim, C. S. Hong, S.-H. Bae, C. Qin, J. Wei, G. Wang, and Y. Yang, "DeepSeek Models: A Comprehensive Survey of Methods and Applications," *TechRxiv*, 2025. doi: 10.36227/techrxiv.174198511.15158242.

[15] S. Shekhar, T. Dubey, K. Mukherjee, A. Saxena, A. Tyagi, and N. Kotla, "Towards Optimizing the Costs of LLM Usage," *arXiv preprint arXiv:2402.01742*, 2024. doi: 10.48550/arXiv.2402.01742.